

Module-4

Client-side Scripting Language

Client-Side Programming

- HTML is good for developing *static* pages
 - can specify **text/image layout, presentation, links, ...**
 - Web page looks the **same each time it is accessed**
 - in order to develop interactive/reactive pages, must integrate programming in some form or another
- **Client-side programming**
 - programs are **written in a separate programming (or scripting) language**
e.g., JavaScript, JScript, VBScript
 - programs are embedded in the HTML of a Web page, with (HTML) tags to identify the program component
e.g., **<script type="text/javascript"> ... </script>**

Javascript

The `<script>` Tag:

```
<script>
```

```
document.getElementById("demo").innerHTML = "My First JavaScript";
```

```
</script>
```

- place any number of scripts in an HTML document.
- Scripts can be placed in the `<body>`, or in the `<head>` section of an HTML page, or in both.

External JavaScript

- External scripts are practical when the **same code is used in many different web pages.**
- JavaScript files have the file extension **.js.**
- To use an external script, put the name of the script file in the src (source) attribute of a <script> tag:

<script src="myScript.js"></script>
- Can place an external script reference in <head> or <body> as you like.

External JavaScript Advantages

- Placing scripts in external files has some advantages:
 - It separates HTML and code
 - It makes HTML and JavaScript easier to read and maintain
 - Cached JavaScript files can speed up page loads
- To add several script files to one page - use several script tags:

```
<script src="myScript1.js"></script>
```

```
<script src="myScript2.js"></script>
```

External References

- An external script can be referenced in 3 different ways:
 - With a full URL (a full web address)
 - With a file path (like /js/)
 - Without any path

- **full URL :**

```
script src="https://www.javascript.com/js/myScript.js"></script>
```

- **file path :**

```
<script src="/js/myScript.js"></script>
```

- **no path :**

```
<script src="myScript.js"></script>
```

JavaScript Output

JavaScript can "display" data in different ways:

- Writing into an HTML element, using `innerHTML`.
- Writing into the HTML output using `document.write()`.
- Writing into an alert box, using `window.alert()`.
- Writing into the browser console, using `console.log()`.

Using innerHTML

- To access an HTML element, JavaScript can use the **document.getElementById(id) method**
- The id attribute defines the HTML element.
- The innerHTML property defines the HTML content:

```
<!DOCTYPE html>
<html>
<body>
<h2>My First Web Page</h2>
<p>My First Paragraph.</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>
</body>
</html>
```

My First Web Page

My first paragraph.

Try it

Using document.write()

- For testing purposes, it is convenient to use **document.write()**
- Never call document.write after the document has finished loading.
- It will overwrite the whole document.

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My first paragraph.</p>

<button type="button" onclick="document.write(5 + 6)">
Try it
</button>

</body>
</html>
```

My First Web Page

My first paragraph.

Try it

11

Using window.alert()

- can use an alert box to display data

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>My First Web Page</h2>
```

```
<p>My first paragraph.</p>
```

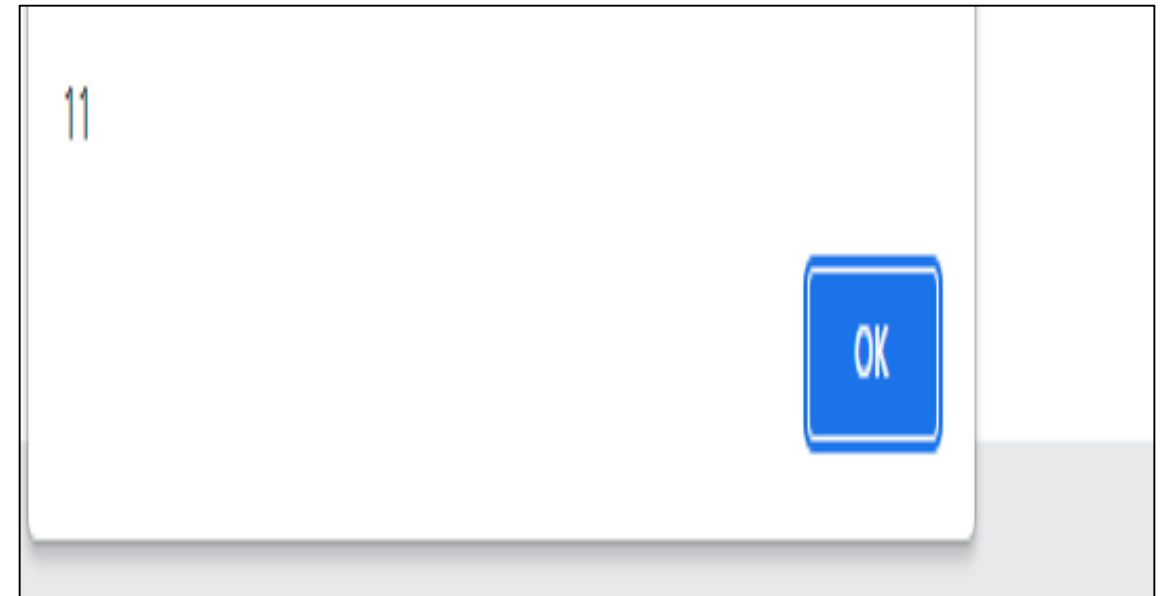
```
<script>
```

```
window.alert(5 + 6);
```

```
</script>
```

```
</body>
```

```
</html>
```



- Skip the window keyword.
- In JavaScript, the window object is the global scope object, that means that variables, properties, and methods by default belong to the window object.
- This also means that specifying the window keyword is optional:

```
<script>
```

```
  alert(5 + 6);
```

```
</script>
```

Using console.log()

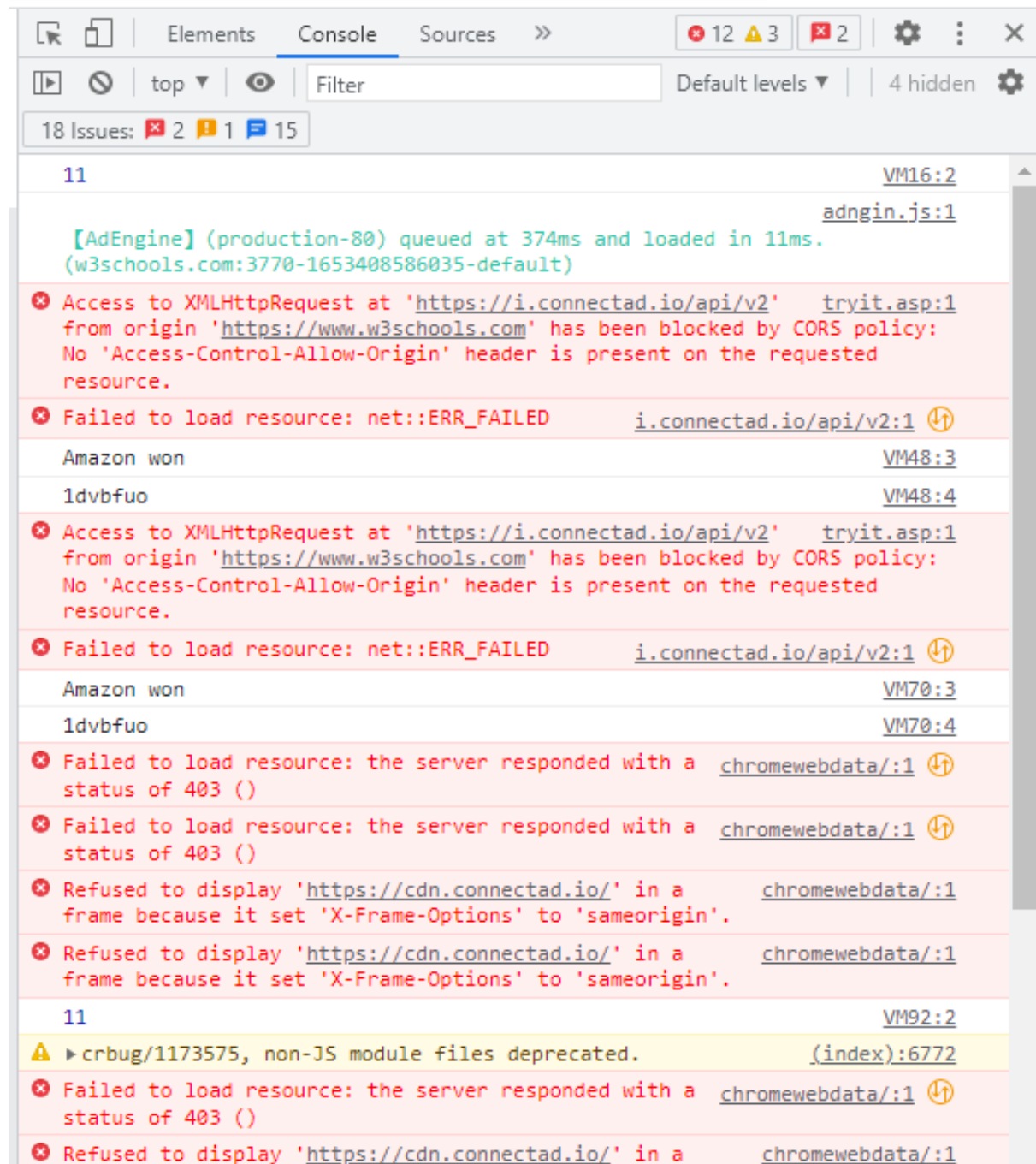
```
<!DOCTYPE html>
<html>
<body>

<h2>Activate Debugging</h2>

<p>F12 on your keyboard will activate
debugging.</p>
<p>Then select "Console" in the debugger
menu.</p>
<p>Then click Run again.</p>

<script>
console.log(5 + 6);
</script>

</body>
</html>
```



JavaScript Print

- JavaScript does not have any print object or print methods.
- Cannot access output devices from JavaScript.
- The only exception is that you can call the `window.print()` method in the browser to print the content of the current window.

JavaScript Statements

JavaScript Programs

- A **computer program** is a list of "instructions" to be "executed" by a computer.
- In a programming language, these programming instructions are called **statements**.
- A **JavaScript program** is a list of programming **statements**.
- In HTML, JavaScript programs are executed by the web browser.

JavaScript Statements

- JavaScript statements are composed of:
 - Values, Operators, Expressions, Keywords, and Comments.
 - Most JavaScript programs contain many JavaScript statements.
 - The statements are executed, one by one, in the same order as they are written.
 - JavaScript programs (and JavaScript statements) are often called JavaScript code.
 - Semicolons separate JavaScript statements.

```
let a, b, c;  
a = 5;  
b = 6;  
c = a + b;
```

JavaScript White Space

- JavaScript ignores multiple spaces.
- Can add white space to your script to make it more readable.
- The following lines are equivalent:

```
let person = "Hege";
```

```
let person="Hege";
```


JavaScript Code Blocks

- JavaScript statements can be grouped together in code blocks, inside curly brackets {...}.
- The purpose of code blocks is to define statements to be executed together.

```
function myFunction()  
  
{  
    document.getElementById("demo1").innerHTML = "Hello!";  
    document.getElementById("demo2").innerHTML = "How are you?";  
}
```

JavaScript Keywords

- JavaScript statements often start with a **keyword** to identify the JavaScript action to be performed.
- JavaScript keywords are reserved words. Reserved words cannot be used as names for variables.

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

JavaScript Comments

- Not all JavaScript statements are "executed".
- Code after double slashes `//` or between `/*` and `*/` is treated as a comment.
- Comments are ignored, and will not be executed.

JavaScript Syntax

- JavaScript syntax is the set of rules, how JavaScript programs are constructed

JavaScript Values:

- The JavaScript syntax defines two types of values:
 - Fixed values
 - Variable values
- Fixed values are called **Literals**.
- Variable values are called **Variables**.

JavaScript Literals

The two most important syntax rules for fixed values are:

1. **Numbers** are written with or without decimals

10.50

1001

2. **Strings** are text, written within double or single quotes

"John Doe"

'John Doe'

JavaScript Identifiers / Names

- Identifiers are JavaScript names.
- Identifiers are used to name variables and keywords, and functions.
- The rules for legal names are the same in most programming languages.
- A JavaScript name must begin with:
 - A letter (A-Z or a-z)
 - A dollar sign (\$)
 - Or an underscore (_)
- Subsequent characters may be letters, digits, underscores, or dollar signs.

JavaScript is Case Sensitive

- All JavaScript identifiers are case sensitive.
- The variables `lastName` and `lastname`, are two different variables.

JavaScript and Camel Case

Programmers have used different ways of joining multiple words into one variable name:

- **Hyphens:**
 - `first-name`, `last-name`, `master-card`, `inter-city`.
 - Hyphens are not allowed in JavaScript. They are reserved for subtractions.
- **Underscore:** `first_name`, `last_name`, `master_card`, `inter_city`.
- **Upper Camel Case (Pascal Case):** `FirstName`, `LastName`, `MasterCard`, `InterCity`.
- **Lower Camel Case:** JavaScript programmers tend to use camel case that starts with a lowercase letter:
 - `firstName`, `lastName`, `masterCard`, `interCity`.

JavaScript Character Set

- JavaScript uses the **Unicode** character set.
- Unicode covers (almost) all the characters, punctuations, and symbols in the world.

JavaScript Variables

What are Variables?

- Variables are containers for storing data (storing data values).
- In a programming language, variables are used to store data values.
- JavaScript uses the keywords `var`, `let` and `const` to declare variables.
- An equal sign is used to assign values to variables.

```
let x;
```

```
x = 6;
```

JavaScript Variables

4 Ways to Declare a JavaScript Variable:

- Using var
- Using let
- Using const
- Using nothing

- Always declare JavaScript variables with var, let, or const.
- The var keyword is used in all JavaScript code from 1995 to 2015.
- The let and const keywords were added to JavaScript in 2015.
- code to run in older browser, must use var.
- a general rule: always declare variables with const.
- the value of the variable can change, use let.

Re-Declaring JavaScript Variables

- re-declare a JavaScript variable declared with var, it will not lose its value.

```
var carName = "Volvo";  
var carName;
```

- cannot re-declare a variable declared with let or const.

- will not work:

```
let carName = "Volvo";  
let carName;
```

- If you put a number in quotes, the rest of the numbers will be treated as strings, and concatenated.

```
let x = 2 + 3 + "5";
```

Result : 55

JavaScript Let

- The let keyword was introduced in ES6 (2015).
- Variables defined with let cannot be Redeclared.
- Variables defined with let must be Declared before use.
- Variables defined with let have Block Scope.

Cannot be Redeclared

```
let x = "John Doe";
```

```
let x = 0;
```

```
// SyntaxError: 'x' has already  
    been declared
```

With var you can redeclare

```
var x = "John Doe";
```

```
var x = 0;
```

Block Scope

- Before 2015, JavaScript had only Global Scope and Function Scope.
- Two important new JavaScript keywords: `let` and `const` were introduced.
- These two keywords provide Block Scope in JavaScript.
- Variables declared inside a `{ }` block cannot be accessed from outside the block

Example

```
{  
  let x = 2;  
}  
// x can NOT be used here
```

Variables declared with the var keyword can NOT have block scope.

```
{  
  var x = 2;  
}  
// x CAN be used here
```

Redeclaring Variables

- Redeclaring a variable using the var keyword can impose problems.
- Redeclaring a variable inside a block will also redeclare the variable outside the block

```
var x = 10;  
// Here x is 10  
{  
  var x = 2;  
  // Here x is 2  
}  
// Here x is 2
```


- Redeclaring a variable using the let keyword can solve this problem.
- Redeclaring a variable inside a block will not redeclare the variable outside the block

```
let x = 10;  
// Here x is 10  
{  
  let x = 2;  
  // Here x is 2  
}  
// Here x is 10
```

- Redeclaring a variable with let, in another block, IS allowed

```
let x = 2;  // Allowed
```

```
{  
let x = 3;  // Allowed  
}
```

```
{  
let x = 4;  // Allowed  
}
```

JavaScript Const

- The const keyword was introduced in ES6 (2015).
- Variables defined with const cannot be Redeclared.
- Variables defined with const cannot be Reassigned.
- Variables defined with const have Block Scope.

Cannot be Reassigned

- A const variable cannot be reassigned

```
const PI = 3.141592653589793;
```

```
PI = 3.14;    // This will give an error
```

```
PI = PI + 10; // This will also give an error
```

Must be Assigned

- JavaScript const variables must be assigned a value when they are declared

```
//correct
```

```
const PI = 3.14159265359;
```

```
//Incorrect
```

```
const PI;
```

```
PI = 3.14159265359;
```

Block Scope

- Declaring a variable with const is similar to let when it comes to Block Scope.
- The x declared in the block, in this example, is not the same as the x declared outside the block

```
const x = 10;
```

```
// Here x is 10
```

```
{
```

```
const x = 2;
```

```
// Here x is 2
```

```
}
```

```
// Here x is 10
```

Task-1

Create a Hello World Web Page using various JavaScript Output elements